NOT ON RECORD

Alumna: Bianca Ana María Domozina

Grupo: 1.º DAW

Módulo: Programación - 1.ª evaluación

Curso: 2026-2027

Repositorio GitHub: https://github.com/BiancaAMD/NotOnRecord

Índice

1. Descripción del juego . . . 2

1.1 Objetivo . . . 2

1.2 Reglas y datos del juego . . . 2

1.3 Desarrollo de la partida . . . 2

Ejemplo de turno . . . 3

1.4 Esquema del flujo del juego . . . 3

1.5 Decisiones y duelo . . . 4

1.6 Puntuación, victoria y derrota . . . 4

2. Funcionalidades implementadas y diferencias con la propuesta inicial . . . 4

Por qué se añadió el duelo y cómo encaja . . . 5

Decisiones de diseño respecto a la propuesta . . . 5

Cambios hechos para corregir problemas técnicos . . . 5

3. Diseño del programa . . . 6

3.1 Constantes . . . 6

3.2 Variables . . . 6

3.3 Operadores y entrada/salida . . . 6

3.4 Estructuras de control . . . 7

3.5 Uso de arrays unidimensionales . . . 7

3.6 Fragmento clave: el duelo . . . 8

3.7 Aleatoriedad y probabilidades . . . 8

4. Pruebas . . . 9

4.1 Cómo se han realizado . . . 9

4.2 Pruebas normales . . . 10

4.3 Pruebas con datos incorrectos . . . 11

4.4 Casos límite . . . 12

5. Dificultades encontradas, soluciones y conclusión . . . 13

5.1 Dificultades durante el desarrollo . . . 13

5.2 Correcciones tras la revisión del profesor . . . 13

5.3 Conclusión personal . . . 14

5.4 Ampliaciones previstas para la 2.ª evaluación (no implementadas) . . . 14

Anexo: capturas de pantalla . . . 15

A. Capturas de apariencia . . . 15

B. Evidencias de pruebas normales y con datos incorrectos . . . 15

C. Evidencias de decisiones, duelo y casos límite . . . 18

1. Descripción del juego

1.1 Objetivo

Not On Record es un juego por consola en el que el jugador es el portero de un edificio. A lo largo de una jornada llegan visitantes que dicen ser residentes y piden que se les abra. El jugador debe investigar si el visitante es quien dice ser o un impostor, y decidir si lo deja pasar o lo rechaza. El juego está inspirado en *That's Not My Neighbor*: no todo lo que parece sospechoso es culpable y la información puede no ser fiable, porque el visitante puede no entregar el documento y el conviviente puede no estar en casa.

El objetivo del jugador es acertar con el mayor número posible de visitantes sin llegar a cometer 3 errores y sin quedarse sin salud si un impostor le ataca.

1.2 Reglas y datos del juego

El edificio tiene 5 residentes repartidos en 3 pisos; en dos de ellos viven dos personas (convivientes) y en el otro vive una sola:

Residente	Piso	DNI	Teléfono del piso
Julián Ramírez	3	K23O45P	3030
Karla Real	3	O13234D	3030
Sonia Doñoro	2	W24509U	2020
Fran Pardo	2	L96436N	2020
Cristina Casado	1	K34424F	1010 (vive sola)

Las reglas que rigen cada turno son estas:

- Selección del visitante. La partida tiene 5 turnos, pero en cada turno el residente se sortea de forma independiente entre las 5 posiciones del registro. Por tanto un mismo residente puede aparecer en varios turnos y otro puede no aparecer nunca; los 5 turnos no corresponden a un residente cada uno.
- Impostor o residente. Con un 50 % de probabilidad el visitante es un residente real y con otro 50 % un impostor.
- Tipos de impostor. Un impostor altera un dato de su documento: el nombre, el piso o el DNI (se le antepone una letra). El cuarto tipo es el impostor perfecto: su documento coincide con el registro y solo se le descubre llamando al apartamento, porque el conviviente contesta que el residente sí está en casa y un residente verdadero no estaría llamando a la puerta. En ese caso el conviviente está siempre en casa.
- Excepción del impostor perfecto. Si el residente suplantado vive solo (Cristina Casado) no hay conviviente que pueda contestar, así que el programa no permite ese tipo: sortea de nuevo entre los tres tipos que alteran el documento. Así todo impostor deja siempre alguna pista.
- Piso falso. El piso falso es un número de 0 a 3 distinto del real. El piso 0 es intencionado: en el edificio no existe, así que sirve como una pista más evidente.
- Información no fiable. El visitante entrega el documento por su cuenta solo el 50 % de las veces, y el conviviente solo está en casa el 50 % de las veces cuando el residente no vive solo.

1.3 Desarrollo de la partida

El flujo es: menú principal (1 Empezar partida, 2 Cómo jugar, 3 Salir) y, al empezar, un bucle de 5 turnos. En cada turno el visitante saluda y puede entregar su documento de forma espontánea. Mientras no se tome una decisión, el jugador ve el menú de investigación:

Opción	Acción	Efecto
1	Consultar registro	Muestra la tabla de residentes (nombre, piso, DNI y teléfono) para comparar con el documento.
2	Pedir documento	Si no lo había entregado, el visitante lo entrega y se muestra en un recuadro tipo DNI. Si ya lo tenía: «Pero... Ya se lo he dado».
3	Llamar al apartamento	Se escribe un teléfono. Solo contesta alguien si el número es el del piso del residente y el conviviente está en casa; si no, «Nadie contesta». Una sola llamada por turno.
4	Tomar decisión	Se elige A (dejar pasar) o B (rechazar). Termina el turno.
0	Abandonar jornada	Termina la partida y se muestra el resumen.

Cualquier otra entrada muestra «Elija una opción válida» y no rompe el programa, porque todas las opciones se leen como texto.

Ejemplo de turno

Turno en el que el visitante es un impostor con el DNI alterado y el jugador investiga antes de decidir (las líneas «Opción» son lo que escribe el jugador):

```
TURNO 2
VISITANTE: Buenos días señor portero, ¿me abre por favor?
*El visitante no le entrega nada*
Opción: 2
PORTERO: Por favor, entrégueme su documento
VISITANTE: Perdona, aquí tiene
+--------------------------------+
|            DOCUMENTO           |
+--------------------------------+
| NOMBRE: Sonia Doñoro           |
| PISO:   2                      |
| DNI:    FW24509U               |
+--------------------------------+
Opción: 1      (en el registro, el DNI real de Sonia es W24509U)
Opción: 4 -> B (rechazar)
PORTERO: ¡Eres un impostor!
*EL IMPOSTOR HA SALIDO CORRIENDO*
ACIERTOS: 1/5
```

El jugador descubre la pista comparando el documento con el registro: el DNI lleva una letra de más delante.

1.4 Esquema del flujo del juego

Fase	Qué ocurre	Siguiente paso
Inicio	Se muestra el título y el menú principal.	Opción 1: empezar. Opción 2: cómo jugar. Opción 3: salir.
Preparación	Se reinician puntos, errores, aciertos, salud (100) y refuerzos (1).	Empieza el bucle de turnos.
Turno	Se sortea residente e impostor, se copian los datos al documento y se busca al conviviente.	El visitante saluda y puede entregar el documento.
Investigación	El jugador usa registro, documento y llamada (según lo permitido).	Toma la decisión (opción 4) o abandona (opción 0).
Resolución	Se compara la decisión con la verdad, se suman puntos o errores y, si procede, hay duelo.	Siguiente turno, o fin si hay 3 errores, derrota en duelo o abandono.
Final	Resumen de puntos, errores y aciertos, y valoración o GAME OVER.	Vuelve al menú principal.

1.5 Decisiones y duelo

Decisión	Realidad	Resultado
A. Dejar pasar	Residente	Acierto: +100 puntos y +1 acierto.
A. Dejar pasar	Impostor	Error: +1 error y penalización 50 x nº de errores.
B. Rechazar	Residente	«Lo siento, no puede pasar». Error: +1 error y penalización creciente.
B. Rechazar	Impostor	«¡Eres un impostor!». Acierto. Con 50 % el impostor huye (+100 puntos) y con 50 % ataca y empieza un duelo.

El mensaje de acusación solo aparece cuando el visitante es realmente un impostor; si se rechaza a un residente, el portero se limita a negar la entrada.

El duelo es un combate por turnos. El portero y el impostor empiezan con 100 de salud; la salud del portero se conserva entre duelos de la misma partida y se reinicia al empezar una nueva. En cada ronda el jugador elige:

Acción	Efecto
1. Atacar	Daño aleatorio entre 1 y 50 al impostor.
2. Bloquear ataque	El golpe del impostor en esa ronda no hace daño. **No se puede bloquear dos veces seguidas**: el segundo intento avisa y el impostor ataca con normalidad.
3. Pedir refuerzos	Reducen al impostor al instante, pero solo hay 1 por partida. Cuando se agotan aparece «no quedan refuerzos», la acción no hace nada y el impostor contraataca.
Otra entrada	«Opción no válida, pierdes el turno»: el impostor contraataca.

Si tras la acción del jugador el impostor sigue en pie, contraataca con un daño aleatorio entre 1 y 50 (salvo que haya bloqueado). Si el jugador reduce al impostor, cuenta como acierto; si el jugador se queda sin salud, cuenta como error con su penalización y la partida termina.

1.6 Puntuación, victoria y derrota

- Puntos: +100 por acierto; en cada error se resta 50 x nº de errores acumulados (50, 100, 150...). La puntuación puede ser negativa.
- Derrota por errores: al llegar a 3 errores sale «HAS COMETIDO DEMASIADOS ERRORES» y la partida termina con GAME OVER. El portero sigue con salud: pierde la jornada, no la vida.
- Derrota en duelo: si la salud del portero llega a 0 sale «El impostor te ha derrotado» y la partida termina con GAME OVER. Si esa derrota es además el tercer error, solo se muestra este mensaje.
- Fin normal: se completan los 5 turnos o el jugador abandona. Valoración final según puntos: EXPERTO (400 o más), PROFESIONAL (200 o más) y NOVATO (menos de 200).
- El resumen final muestra puntos, errores (x/3) y aciertos (x/5).

2. Funcionalidades implementadas y diferencias con la propuesta inicial

Todo el programa está en un único método `main` de la clase `NotOnRecord`, como se pide en la primera evaluación. La columna «Estado» se ha revisado con la versión entregada.

Funcionalidad	Estado	Comentario
Menú principal (empezar, cómo jugar, salir)	Hecho	do-while con switch sobre texto. «Cómo jugar» explica las opciones, las reglas y el duelo.
Registro de residentes en arrays paralelos	Hecho	nombres, pisos, dnis y telefonos.

Funcionalidad	Estado	Comentario
Visitante aleatorio y generación del impostor	Hecho	4 tipos de impostor mediante Math.random(); los residentes pueden repetirse.
Documento en recuadro tipo DNI	Hecho	Se entrega solo (50 %) o al pedirlo; no se puede pedir dos veces.
Llamada al apartamento con conviviente	Hecho	Busca al conviviente en el array de pisos; una llamada por turno.
Decisión A/B con validación	Hecho	Repite hasta que se escribe A o B (sin distinguir mayúsculas).
Duelo con salud, bloqueo limitado y refuerzos	Hecho	Se activa al rechazar a un impostor que decide atacar.
Puntuación, errores y valoración final	Hecho	Penalización creciente; dos finales de derrota con mensajes distintos.
Colores ANSI	Hecho	Constantes de texto para color y negrita.
Pistas apuntadas, preguntas y aviso a seguridad	No hecho	Eliminadas respecto a la propuesta inicial (ver diferencias).

Por qué se añadió el duelo y cómo encaja

La propuesta inicial era un juego de investigación sin consecuencias físicas: se leía información y se decidía. Al probar las primeras versiones el juego se hacía monótono, porque casi todo consistía en comparar datos. Por eso se añadió el duelo: rechazar a un impostor tiene riesgo, porque puede huir o atacar. De esta forma la investigación y la acción quedan unidas: lo que se descubre en el turno decide qué se hace, y esa decisión puede acabar en combate. Además el duelo usa variables de estado (salud, daño, refuerzos) y un bucle `while` con varias condiciones de salida.

Decisiones de diseño respecto a la propuesta

- Se eliminaron los arrays de pistas (`pistaActiva`, `pistaDescubierta`) y las opciones «Hacer una pregunta», «Apuntar pista» y «Avisar a seguridad»: eran opciones de solo leer y más complejas. Las pistas salen directamente de comparar el documento, el registro y la llamada.
- Se añadió la figura del conviviente, que obliga a recorrer el registro buscando otro residente del mismo piso.
- Se simplificó la decisión final a dos opciones (dejar pasar o rechazar).
- Se eliminó la constante de número de visitantes: el número de turnos coincide con `NUM_RESIDENTES` (5).

Cambios hechos para corregir problemas técnicos

No son decisiones de diseño sino arreglos, que se explican en el apartado 5: el límite de bloqueos seguidos, que la salud no baje de 0, los mensajes de rechazo y de derrota que se contradecían, y el uso de `abecedario.length` y `REFUERZOS_INICIALES` en lugar de números fijos.

3. Diseño del programa

3.1 Constantes

Constante	Valor	Uso
MAX_ERRORES	3	Errores permitidos antes de perder.
NUM_RESIDENTES	5	Tamaño de los arrays y número de turnos.
PUNTOS_ACIERTO	100	Puntos por cada acierto.
PENALIZACION	50	Se multiplica por el nº de errores.
SALUD_INICIAL	100	Salud de portero e impostor al empezar.
DANO_MAX	50	Daño máximo de un golpe.
REFUERZOS_INICIALES	1	Refuerzos disponibles al empezar cada partida.
RESET, ROJO, VERDE, AMARILLO, AZUL, MAGENTA, CIAN, NEGRITA	códigos ANSI	Colores de la consola.

Todas se declaran con `final` y en MAYÚSCULAS; las variables usan camelCase.

3.2 Variables

Variable	Tipo	Función
puntos, errores, aciertos	int	Estado de la partida.
posResidente, posConviviente	int	Posición del residente elegido y de su conviviente (-1 si vive solo).
esImpostor, convivienteEnCasa	boolean	Verdad oculta del visitante y si alguien contesta al teléfono.
documentoEntregado, llamadaRealizada	boolean	Evitan repetir la entrega del documento y la llamada en un turno.
decidido, partidaTerminada	boolean	Controlan el fin del turno y el fin de la partida.
opcion, opcionPrincipal, decision, accionDuelo	String	Entradas del jugador, leídas como texto.
numeroTelefono	String	Teléfono escrito en la llamada. Es `String` porque no se hacen operaciones con él y así se compara con `equals`.
saludPortero, saludImpostor, dano, refuerzosRestantes	int	Estado del duelo.
haBloqueadoAnterior, bloqueando	boolean	Límite de bloqueo: recuerdan si se bloqueó en la ronda anterior y en la actual.

Qué se reinicia y cuándo. Algunas variables se reinician al empezar la partida y otras en cada turno:

Se reinicia al empezar la partida	Se reinicia en cada turno	Se reinicia en cada duelo / ronda
puntos, errores, aciertos, saludPortero, refuerzosRestantes, partidaTerminada	posResidente, esImpostor, llamadaRealizada, decidido, documentoEntregado, posConviviente, convivienteEnCasa, datosDeclarados	Duelo: saludImpostor y haBloqueadoAnterior. Ronda: bloqueando.

La salud del portero y los refuerzos no se reinician entre duelos: se arrastran durante toda la partida.

3.3 Operadores y entrada/salida

- Aritméticos: `puntos - PENALIZACION * errores`, sumas de aciertos, restas de salud.
- Relacionales y lógicos: `&&`, `||`, `!` (por ejemplo, `saludPortero > 0 && saludImpostor > 0`).
- Entrada: `BufferedReader` con `readLine()`. Todo se lee como `String` y se compara con `equals`, de modo que escribir una letra donde se espera un número no provoca excepciones. Se eligió

`BufferedReader` porque es lo trabajado en clase y devuelve la línea completa.

- Salida: `println` y `printf` con formato (`%-20s`) para alinear tablas y el recuadro del documento.

3.4 Estructuras de control

Estructura	Dónde se usa
do-while	Menú principal, menú de investigación de cada turno y validación de la decisión A/B.
for	Turnos de la jornada (`contador < NUM_RESIDENTES && !partidaTerminada`) y recorridos de arrays.
while	Duelo: se repite mientras ambos tengan salud.
switch	Opciones de los menús, tipo de impostor, entrega del documento, huida o ataque y acciones del duelo.
if / else if / else	Decisiones, resultado del duelo, valoración final y comprobaciones de la llamada.

Fin del turno: las tres condiciones. El bucle de investigación se repite con `while (!opcion.equals("0") && !decidido && partidaTerminada == false)`. Cada condición corta el turno por una razón distinta:

- `!opcion.equals("0")`: el jugador abandona; después se pone `partidaTerminada = true` para cortar también el bucle de turnos.
- `!decidido`: se ha tomado la decisión (A o B), y por tanto el turno ha terminado y se pasa al siguiente visitante.
- `partidaTerminada == false`: la partida acaba dentro del turno (tercer error o derrota en un duelo) y no se debe seguir mostrando el menú.

3.5 Uso de arrays unidimensionales

Array	Tipo y tamaño	Para qué sirve
nombres, pisos, dnis, telefonos	String[NUM_RESIDENTES]	Registro en arrays paralelos: la misma posición describe al mismo residente. El teléfono es del piso y se repite en convivientes.
datosDeclarados	String[3]	Lo que el visitante declara: [0] nombre, [1] piso, [2] DNI. Se copia del registro y se altera si es impostor.
nombresCambiados	String[NUM_RESIDENTES]	Versión alterada de cada nombre (por ejemplo «Fron Pardo») para el impostor del tipo 0.
abecedario	String[7]	Letras que se anteponen al DNI para estropearlo.

¿Por qué cuatro arrays unidimensionales y no uno bidimensional? Una matriz `String[5][4]` obligaría a recordar qué columna es cada dato (0 nombre, 1 piso...) y a convertir todo a `String`. Con arrays paralelos cada dato tiene un nombre claro (`nombres[i]`, `dnis[i]`), el código se lee mejor y es lo trabajado en esta evaluación. La contrapartida es que hay que mantener los cuatro arrays sincronizados (la misma posición en todos). Con `datosDeclarados` pasa lo mismo: tiene exactamente 3 posiciones porque el documento solo muestra 3 datos (nombre, piso y DNI).

Operaciones realizadas sobre los arrays:

a) Recorrido. Se imprime el registro completo recorriendo los cuatro arrays paralelos:

```
for (int i = 0; i < NUM_RESIDENTES; i++) {
    System.out.printf("%-20s| %-5s| %-10s| %-8s |%n",
                      nombres[i], pisos[i], dnis[i], telefonos[i]);
}
```

b) Búsqueda. Se busca al conviviente: otro residente con el mismo piso que no sea el propio visitante. Se parte de -1 (nadie) porque ninguna posición de un array puede valer -1, y así se sabe si vive solo. Sin la

condición `i != posResidente` el visitante se encontraría a sí mismo en su propio piso y sería su propio conviviente:

```
posConviviente = -1;
for (int i = 0; i < NUM_RESIDENTES; i++) {
    if (pisos[i].equals(pisos[posResidente]) && i != posResidente) {
        posConviviente = i;
    }
}
convivienteEnCasa = posConviviente != -1 && Math.random() < 0.5;
```

c) Selección. Se elige un residente al azar y se copian sus datos al array del visitante: `posResidente = (int)(Math.random() * NUM_RESIDENTES);`. El paréntesis es importante: sin él el cast afecta solo a `Math.random()` y el resultado sería siempre 0.

d) Actualización. Si el visitante es impostor se modifica un elemento de `datosDeclarados`. La letra falsa se elige con `abecedario.length`, de modo que el código sigue funcionando si se añaden letras:

```
case 0: datosDeclarados[0] = nombresCambiados[posResidente]; break;
case 1: do { pisoRandom = (int)(Math.random() * 4); }      // 0 a 3
        while (pisoRandom == Integer.parseInt(datosDeclarados[1]));
        datosDeclarados[1] = String.valueOf(pisoRandom); break;
case 2: datosDeclarados[2] = abecedario[(int)(Math.random() * abecedario.length)]
                           + dnis[posResidente]; break;
case 3: convivienteEnCasa = true; break;   // impostor perfecto
```

e) Conteo. El programa no realiza conteo de elementos de un array. `aciertos` y `errores` son contadores de la partida (se incrementan con cada decisión), no cuentan elementos de ningún array.

3.6 Fragmento clave: el duelo

```
haBloqueadoAnterior = false;                    // cada duelo empieza sin bloqueo previo
while (saludPortero > 0 && saludImpostor > 0) {
    boolean bloqueando = false;                 // en cada ronda se parte de "no bloqueo"
    ... // muestra vida y pide la acción (1 atacar, 2 bloquear, 3 refuerzos)
    // case "2": solo bloquea si haBloqueadoAnterior es false
    if (saludImpostor > 0) {                    // el impostor contraataca si sigue en pie
        dano = (int)(Math.random() * DANO_MAX) + 1;
        if (bloqueando) { ... mensaje de ataque bloqueado ... }
        else { saludPortero = saludPortero - dano;
               if (saludPortero < 0) { saludPortero = 0; } }
    }
    haBloqueadoAnterior = bloqueando;           // se recuerda para la ronda siguiente
}
```

- Por qué `bloqueando` se reinicia en cada ronda: el bloqueo solo protege del golpe de *esa* ronda. Si se declarara fuera del `while`, se quedaría en `true` y el jugador estaría protegido para siempre.
- Por qué `haBloqueadoAnterior` se reinicia fuera del bucle: debe recordar lo ocurrido en la ronda anterior, así que no puede reiniciarse dentro. Sí se reinicia al empezar cada duelo.
- Por qué el contraataque lleva `if (saludImpostor > 0)`: el `while` solo comprueba la salud al empezar la ronda; la acción del jugador (atacar o pedir refuerzos) puede haber derrotado ya al impostor, que entonces no debe golpear.
- Salud mínima 0: tras restar el daño se comprueba si la salud es menor que 0 y se deja en 0, tanto la del impostor como la del portero.
- Fin del duelo: el `while` termina cuando una de las dos saludes llega a 0. Después se mira `saludPortero <= 0` para saber quién ha ganado.

3.7 Aleatoriedad y probabilidades

`Math.random()` devuelve un decimal entre 0 (incluido) y 1 (sin llegar a 1). Al multiplicarlo por *n* y truncar con `(int)` se obtiene un entero de 0 a *n*-1:

Qué se sortea	Expresión	Valores posibles
Residente que llega	(int)(Math.random() * NUM_RESIDENTES)	0 a 4.
Impostor o residente	Math.random() < 0.50	50 % cada uno.
Tipo de impostor	(int)(Math.random() * 4)	0 nombre, 1 piso, 2 DNI, 3 documento perfecto (4 tipos).
Tipo si vive solo y sale 3	(int)(Math.random() * 3)	0, 1 o 2: los 3 tipos que alteran el documento.
Piso falso	(int)(Math.random() * 4)	0 a 3, repitiendo si coincide con el real. El 0 es intencionado.
Letra falsa del DNI	abecedario[(int)(Math.random() * abecedario.length)]	Una de las 7 letras del array.
Entrega espontánea del documento	(int)(Math.random() * 2)	50 % lo entrega y 50 % no.
Conviviente en casa	Math.random() < 0.5	50 % si vive acompañado.
Impostor huye o ataca	(int)(Math.random() * 2)	50 % cada uno.
Daño de un golpe	(int)(Math.random() * DANO_MAX) + 1	1 a 50.

El `+ 1` del daño evita golpes de 0 puntos: sin él, `(int)(Math.random() * 50)` da de 0 a 49; con él da de 1 a 50.

4. Pruebas

4.1 Cómo se han realizado

Las pruebas se hicieron ejecutando el programa y comparando la salida con lo esperado. Cada caso indica la figura del anexo donde se ve la salida real, y cada figura indica a qué caso pertenece.

Casos aleatorios. Algunas situaciones dependen del azar (qué tipo de impostor sale, si el conviviente está en casa, si el impostor huye o ataca). Para comprobarlas se repitió la ejecución con las mismas entradas hasta que salía cada situación. Para forzar un caso concreto se cambiaron temporalmente algunos valores del código (por ejemplo `esImpostor = true` o `REFUERZOS_INICIALES = 0`) y después se volvieron a dejar como estaban. En los casos que dependen de la puntuación, como la valoración EXPERTO (400 puntos o más), se enseña el resumen real.

4.2 Pruebas normales

Nº	Caso de prueba	Resultado esperado	Resultado obtenido	Estado
CP01	Menú principal: 1	Empieza la partida y aparece el turno 1.	Aparece «TURNO 1», el saludo «¿me abre por favor?» y el menú de opciones. (Fig. 13)	Superado
CP02	Menú principal: 2	Muestra «Cómo jugar» y vuelve al menú.	Se muestra el título, las opciones del turno, las reglas y el funcionamiento del duelo. (Fig. 14)	Superado
CP03	Menú principal: 3	Imprime «Saliendo del juego.» y termina.	Se imprime el mensaje y el programa termina.	Superado
CP04	Turno: opción 1	Muestra el registro con los 5 residentes.	Tabla alineada con nombre, piso, DNI y teléfono de los 5 residentes. (Fig. 3)	Superado
CP05	Turno: opción 2 sin documento	El visitante lo entrega y se ve en el recuadro.	«Perdona, aquí tiene» y el documento en el recuadro con nombre, piso y DNI. (Fig. 2)	Superado
CP06	Llamada con el teléfono del piso, impostor y conviviente en casa	Responde el conviviente: «Sí, aquí está».	Contesta Fran Pardo: «¿Diga?» y, al preguntar por Sonia Doñoro, «Sí, aquí está.». (Fig. 4)	Superado
CP07	Llamada con el teléfono del piso, residente real y conviviente en casa	Responde el conviviente: «No, ha salido».	Contesta Julián Ramírez y dice «No, ha salido.» sobre Karla Real. (Fig. 5)	Superado
CP08	Decisión A con un residente	«Has acertado», +100 puntos y aciertos +1.	«Has acertado, era un residente» y «ACIERTOS: 1/5». (Fig. 6)	Superado
CP09	Decisión B con un impostor (huye)	«¡Eres un impostor!», «El impostor ha salido corriendo» y +100 puntos.	«¡Eres un impostor!», «EL IMPOSTOR HA SALIDO CORRIENDO» y «ACIERTOS: 1/5». (Fig. 16)	Superado
CP10	Decisión B con un impostor (ataca) y ganar el duelo	«Has reducido al impostor», +100 puntos y pasa al turno siguiente.	El duelo termina con «Has reducido al impostor» y aparece «TURNO 2». (Fig. 8)	Superado
CP11	Duelo: acción 3 (refuerzos)	El impostor queda reducido al instante y los refuerzos restantes bajan en 1.	«han llegado los refuerzos y han reducido al impostor» y «Has reducido al impostor»: el duelo termina al instante. (Fig. 18)	Superado
CP12	Completar los 5 turnos con 400 puntos o más	Resumen con la valoración EXPERTO.	«Puntos: 500, Errores: 0/3, Aciertos: 5/5, Valoración: EXPERTO». (Fig. 9)	Superado
CP13	Decisión A con un impostor	«Has fallado, era un impostor», +1 error.	«Has fallado, era un impostor» y «ERRORES: 1/3». (Fig. 7)	Superado
CP14	Decisión B con un residente	«Lo siento, no puede pasar», «Has fallado, era un residente» y +1 error.	«PORTERO: Lo siento, no puede pasar», «Has fallado, era un residente» y «Errores: 1/3». (Fig. 15)	Superado

4.3 Pruebas con datos incorrectos

Nº	Caso de prueba	Resultado esperado	Resultado obtenido	Estado
CP15	Menú principal: 9 o una letra	«Opción no válida» y vuelve al menú, sin excepción.	Con «4» sale «Opción no válida» y se repite el menú. (Fig. 10)	Superado
CP16	Menú de turno: 5 o una letra	«Elija una opción válida» y se repite el menú.	Con «5» sale «Elija una opción válida» y se vuelve a mostrar el menú. (Fig. 11)	Superado
CP17	Decisión: «S» o vacío	«Tome una decisión válida» y vuelve a pedirla.	Con «S» sale el aviso en rojo y se pide otra vez A o B. (Fig. 12)	Superado
CP18	Decisión: «a» minúscula	Se acepta igual que «A».	Con «a» se deja pasar al visitante igual que con «A». (Fig. 7)	Superado
CP19	Llamada con un teléfono que no es del piso	«Nadie contesta».	Al escribir el teléfono «4» sale «Nadie contesta». (Fig. 22)	Superado
CP20	Segunda llamada en el mismo turno	«Ya ha llamado».	Al elegir otra vez la opción 3 sale «Ya ha llamado». (Fig. 22)	Superado
CP21	Pedir documento por segunda vez	«Pero... Ya se lo he dado».	El visitante dice «Pero... Ya se lo he dado» y «le señala el documento». (Fig. 24)	Superado
CP22	Duelo: acción 9 o una letra	«Opción no válida, pierdes el turno» y el impostor contraataca.	Con «9» sale «opción no válida, pierdes el turno», el impostor golpea (-29) y la vida baja a 30/100. (Fig. 20)	Superado

4.4 Casos límite

Nº	Caso de prueba	Resultado esperado	Resultado obtenido	Estado
CP23	Duelo: pedir refuerzos cuando ya no quedan	«No quedan refuerzos», no se reduce al impostor y este contraataca.	«no quedan refuerzos»; el impostor sigue con 100/100 y golpea (-41). (Fig. 19)	Superado
CP24	Duelo: bloquear	«Has bloqueado el ataque» y daño recibido 0.	«te proteges del siguiente golpe» y «has bloqueado el ataque del impostor»; la vida se queda en 100/100. (Fig. 17)	Superado
CP25	Duelo: bloquear dos veces seguidas	Aviso «no puedes bloquear dos veces seguidas» y el impostor golpea.	Al repetir «2» sale el aviso, el impostor golpea (-41) y la vida baja a 59/100. (Fig. 17)	Superado
CP26	Perder el duelo (salud 0)	«El impostor te ha derrotado», +1 error, penalización y GAME OVER. Sin mensaje de 3 errores.	Sale «El impostor te ha derrotado», el resumen con GAME OVER y vuelve el menú principal. (Fig. 27)	Superado
CP27	Cometer el tercer error con decisiones	«HAS COMETIDO DEMASIADOS ERRORES» y GAME OVER en el resumen.	«HAS COMETIDO DEMASIADOS ERRORES», resumen con Errores 3/3, Puntos -200 y GAME OVER. (Fig. 21)	Superado
CP28	Primer y segundo error	Penalizaciones de 50 y 100 puntos (total -150).	Con 3 aciertos y 2 errores: 300 - 50 - 100 = 150 puntos, «Errores: 2/3». (Fig. 26)	Superado
CP29	Opción 0 en el primer turno	«Has abandonado la jornada» y resumen con 0 puntos.	«Has abandonado la jornada» y resumen con Puntos 0, Errores 0/3, Aciertos 0/5 y valoración NOVATO. (Fig. 25)	Superado
CP30	Visitante que vive solo (Cristina Casado, piso 1)	posConviviente es -1, no hay excepción y la llamada da «Nadie contesta».	Documento de Cristina Casado (piso 1); al llamar al 1010 sale «Nadie contesta» y no hay excepción. (Fig. 23)	Superado
CP31	Impostor de tipo piso	El piso mostrado nunca coincide con el real y puede ser el 0.	Se obtuvo un documento de Julián Ramírez con PISO 0 (el real es 3). (Fig. 6)	Superado
CP32	Empezar una segunda partida sin cerrar el programa	Puntos, errores, aciertos, salud y refuerzos se reinician.	Tras el resumen vuelve el menú principal y al elegir 1 aparece «TURNO 1». (Fig. 9)	Superado

Los casos CP06, CP07, CP19, CP20 y CP30 cubren las ramas de la llamada (conviviente en casa, residente o impostor, número incorrecto, repetir llamada y vivir solo); CP26 y CP27 distinguen los dos finales de derrota.

5. Dificultades encontradas, soluciones y conclusión

5.1 Dificultades durante el desarrollo

Dificultad	Cómo se detectó	Solución y comprobación
Un cast mal colocado, `(int) Math.random() * 2`, siempre daba 0.	El visitante entregaba siempre el documento (siempre caía el caso 0).	Poner paréntesis: `(int)(Math.random() * 2)`. Se comprobó repitiendo la ejecución hasta ver ambos casos.
Se escribía `if (llamadaRealizada = false)` y asignaba en vez de comparar.	La condición nunca se cumplía y siempre salía «Ya ha llamado».	Usar ==. Se comprobó que la primera llamada del turno funcionaba.
Los puntos y errores se reiniciaban en cada turno.	El resumen final solo mostraba lo del último turno.	Reiniciarlos solo al empezar la partida, fuera del bucle de turnos.
Error «variable might not have been initialized» con la salud en el resumen.	Error de compilación.	Inicializar `saludPortero` al empezar cada partida.
Una letra en el menú provocaba `NumberFormatException`.	Excepción al escribir texto con `Integer.parseInt`.	Leer todo como `String` y usar `switch` con `default`. Se probó con letras y números.
Errores `ArrayIndexOutOfBounds` por tamaños mal calculados.	Excepción en tiempo de ejecución al recorrer o elegir.	Dimensionar los arrays con `NUM_RESIDENTES`.
El turno no terminaba al ganar el duelo.	Tras reducir al impostor seguía el mismo turno.	Marcar `decidido = true` al tomar la decisión y usarlo en la condición del do-while.
Un impostor que vive solo no daba ninguna pista.	Al diseñar el impostor perfecto, que depende del conviviente.	Si sale el tipo perfecto y no hay conviviente, se sortea entre los otros tres tipos.
Bloquear con `dano / 2` hacía el duelo casi imposible de perder.	En las pruebas del duelo.	El bloqueo anula el golpe y, tras la revisión, se limita (apartado 5.2).

5.2 Correcciones tras la revisión del profesor

Problema	Corrección	Cómo se comprobó
Rechazar mostraba «eres un impostor!!» antes de comprobar si lo era.	El mensaje solo sale si es impostor; a un residente se le dice «Lo siento, no puede pasar».	CP09 y CP14.
«HAS MUERTO» al llegar a 3 errores aunque hubiera salud.	Dos finales: «HAS COMETIDO DEMASIADOS ERRORES» (3 errores) y «El impostor te ha derrotado» (sin salud).	CP26 y CP27.
El bloqueo se podía repetir sin límite y el duelo no avanzaba.	Variable `haBloqueadoAnterior`: no se puede bloquear dos veces seguidas.	CP24 y CP25.
La salud podía quedar negativa.	Tras restar el daño, si es menor que 0 se deja en 0 (impostor y portero).	Duelos CP10 y CP26: la vida mostrada nunca baja de 0.
No estaba claro si los residentes se repetían ni el piso 0.	Se documenta que pueden repetirse y que el piso 0 es intencionado.	CP31.
Números fijos en el código.	`abecedario.length` en lugar de 7 y `REFUERZOS_INICIALES` en lugar de 2.	CP11 y CP23.
Mensaje al bloquear.	Se muestra que el ataque ha sido bloqueado en lugar de «te ha golpeado».	CP24.

5.3 Conclusión personal

Lo que más me ha costado ha sido que todas las partes encajen entre sí: los booleanos que controlan el turno, qué variables se reinician y cuándo, y los arrays paralelos. Cuando algo no estaba bien coordinado el juego hacía cosas raras, por ejemplo que el turno no terminara o que los puntos se borraran. También he aprendido a probar los casos límite (un residente que vive solo, quedarse sin refuerzos, escribir una letra en un menú) y a leer con calma los mensajes de error. Me ha servido decidir primero qué quería que pasara en el juego y buscar después cómo programarlo, y prepararme para explicar cada decisión, como por qué uso `-1` o por qué hay arrays paralelos.

5.4 Ampliaciones previstas para la 2.ª evaluación (no implementadas)

Estas ideas no forman parte de la versión entregada, son lo que me gustaría hacer en el siguiente trimestre:

- Modularización: dividir el `main` en métodos (mostrar registro, llamar, duelo, resumen).
- Más jornadas con dificultad creciente.
- Rasgos físicos en el registro (altura, color de pelo...) para que el impostor falle en uno, con un escáner o una opción «observar».
- Lista de visitas esperadas: los residentes avisan de que alguien viene; quien no está en la lista es sospechoso, pero puede ser legítimo.
- Más variedad de personajes (con clases y herencia).
- Reglas del día: al empezar cada jornada sale una norma nueva (por ejemplo, «hoy se rechaza a quien no entregue documento»), guardadas en un array.
- Aviso antes de la multa: el primer error de la jornada solo es un aviso.
- Eventos aleatorios (por confirmar).

Anexo: capturas de pantalla

Las capturas están numeradas y cada una indica el caso de prueba (CP) al que corresponde. Se separan las evidencias de las pruebas de las capturas de apariencia, que solo enseñan el aspecto del juego.

A. Capturas de apariencia



Figura 1. Pantalla de inicio y menú principal. *Evidencia apariencia.*

B. Evidencias de pruebas normales y con datos incorrectos

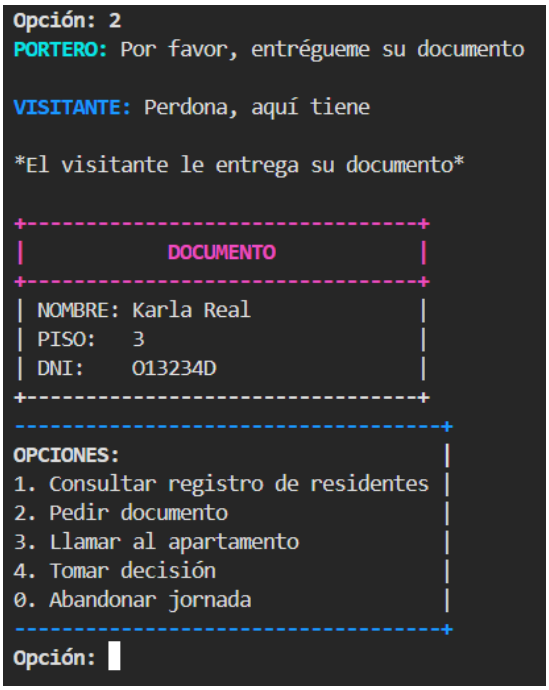


Figura 2. Documento entregado dentro de un recuadro. *Evidencia CP05.*



Figura 3. Registro de residentes. *Evidencia CP04.*

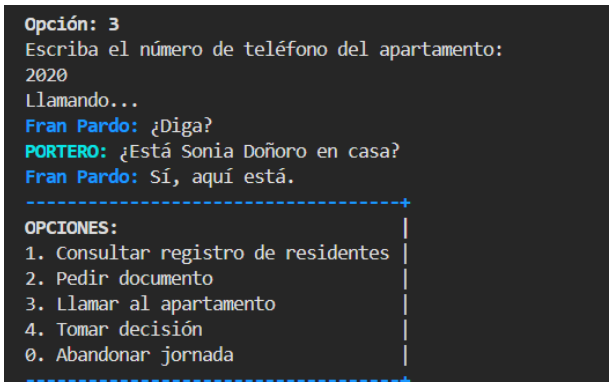


Figura 4. Llamada con conviviente en casa (impostor: «Sí, aquí está»). *Evidencia CP06.*

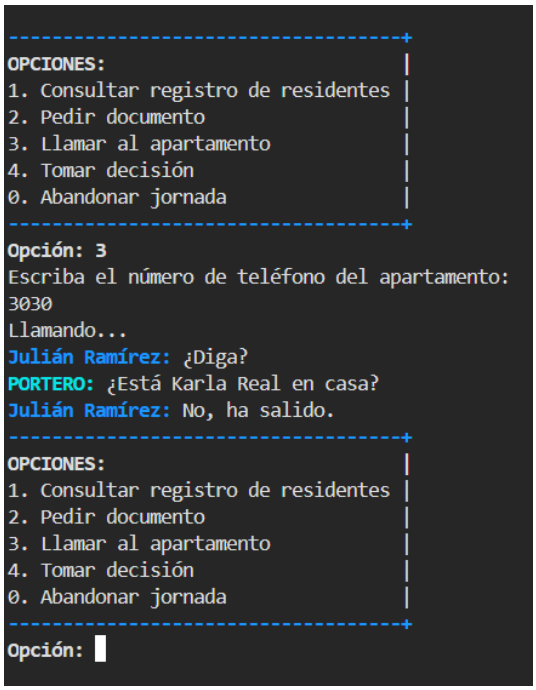


Figura 5. Llamada con conviviente en casa (residente: «No, ha salido»). *Evidencia CP07.*

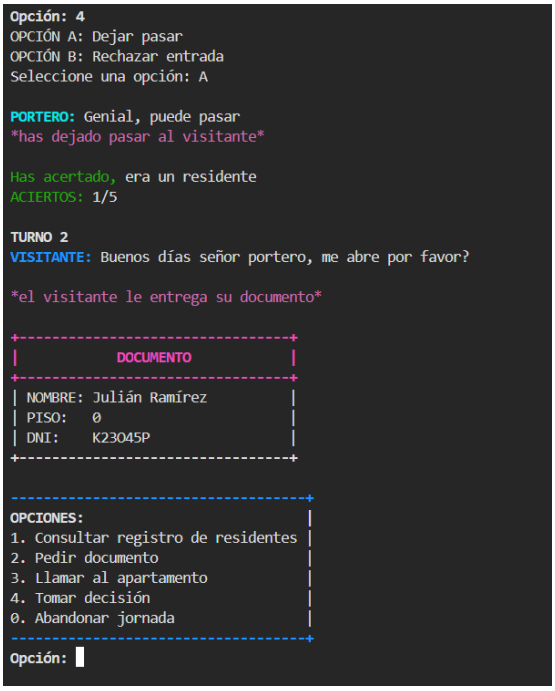


Figura 6. Decisión A con un residente, y siguiente turno con un documento de piso 0. *Evidencia CP08, CP31.*

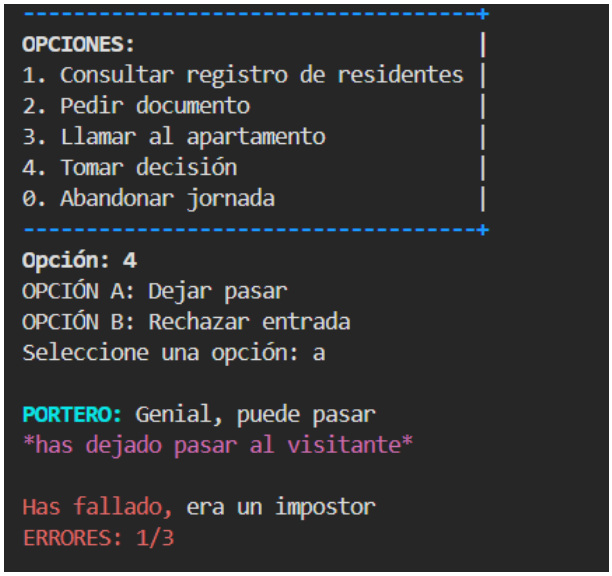


Figura 7. Decisión A con un impostor (se escribe «a» en minúscula). *Evidencia CP13, CP18.*

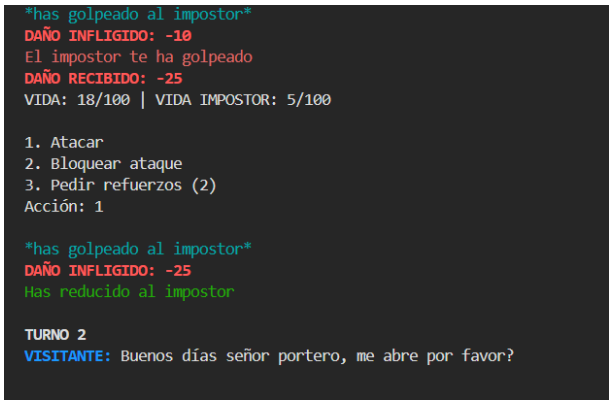


Figura 8. Duelo ganado: «Has reducido al impostor». *Evidencia CP10.*

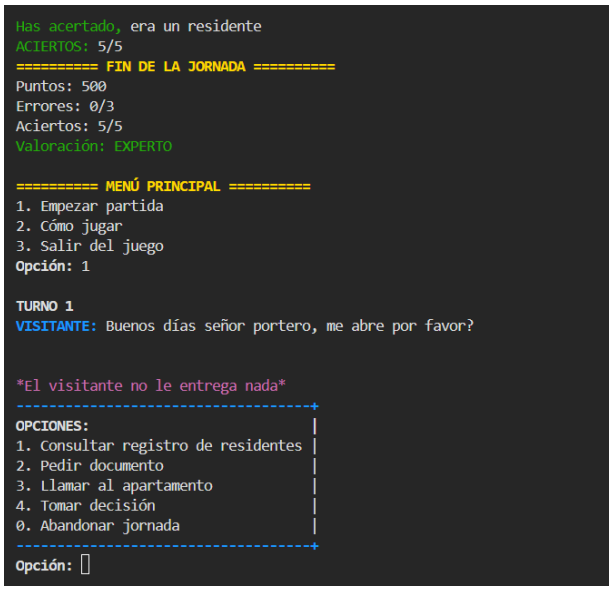


Figura 9. Resumen con valoración EXPERTO y comienzo de una segunda partida. *Evidencia CP12, CP32.*



Figura 10. Menú principal con opción no válida. *Evidencia CP15.*

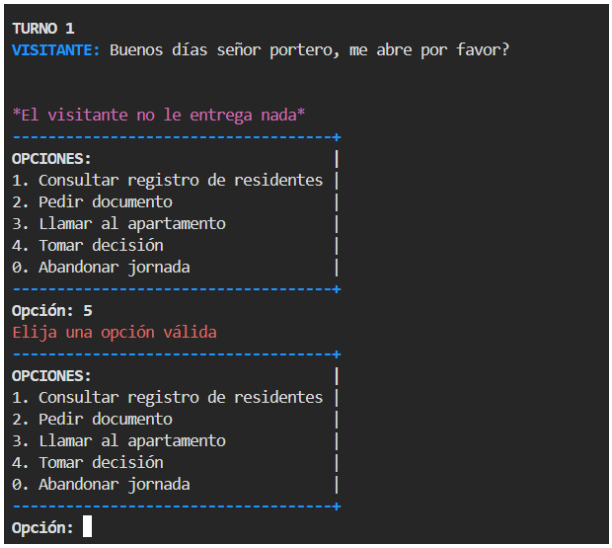


Figura 11. Menú de turno con opción no válida. *Evidencia CP16.*

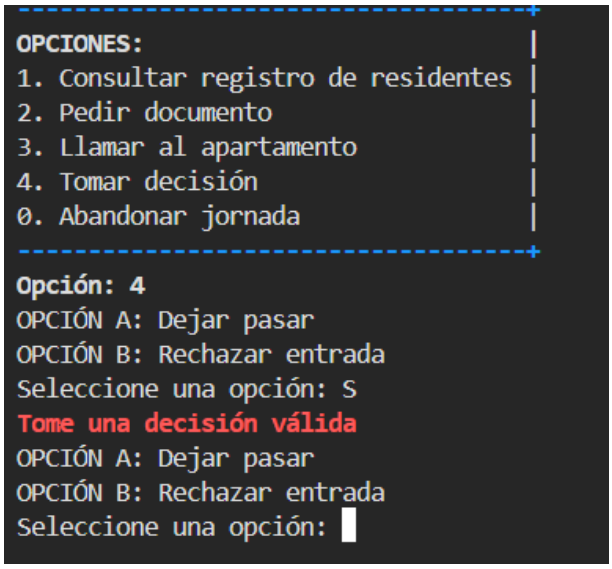


Figura 12. Decisión con una entrada no válida. *Evidencia CP17.*

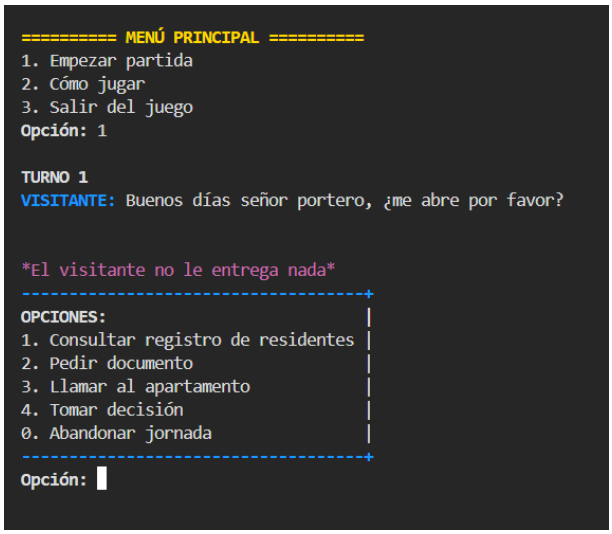


Figura 13. Inicio de la partida: turno 1. *Evidencia CP01.*



Figura 14. Pantalla «Cómo jugar». *Evidencia CP02.*

C. Evidencias de decisiones, duelo y casos límite

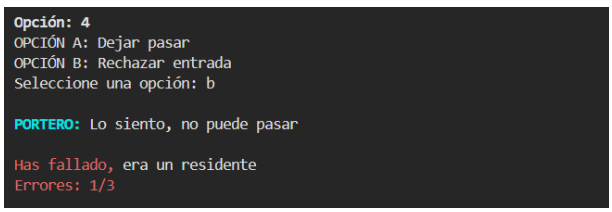


Figura 15. Rechazar a un residente: «Lo siento, no puede pasar». *Evidencia CP14.*

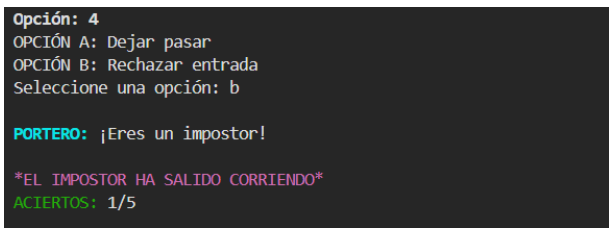


Figura 16. Rechazar a un impostor que huye. *Evidencia CP09.*

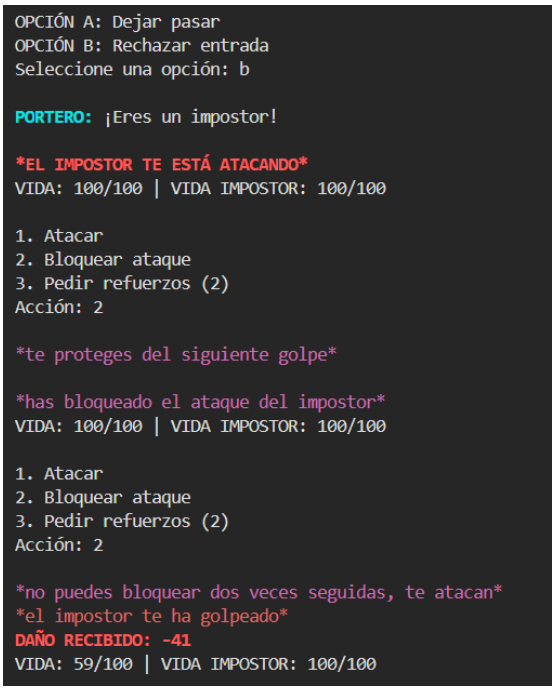


Figura 17. Duelo: bloqueo y bloqueo repetido. *Evidencia CP24, CP25.*

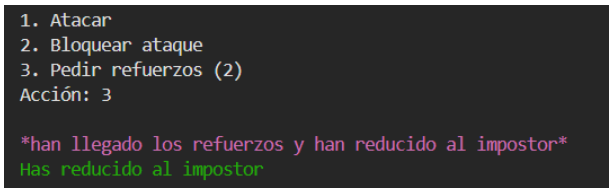


Figura 18. Duelo: refuerzos. *Evidencia CP11.*

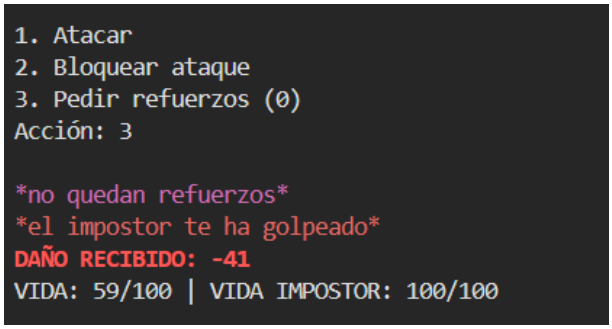


Figura 19. Duelo: sin refuerzos. *Evidencia CP23.*

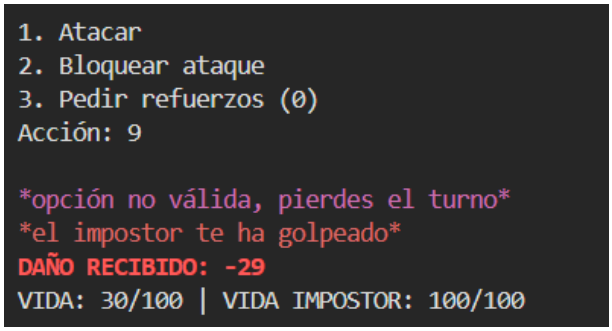


Figura 20. Duelo: acción no válida. *Evidencia CP22.*

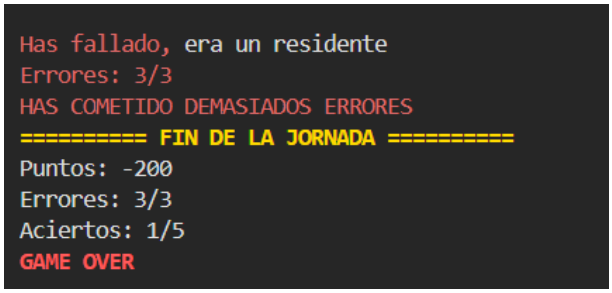


Figura 21. Tres errores: «HAS COMETIDO DEMASIADOS ERRORES». *Evidencia CP27.*

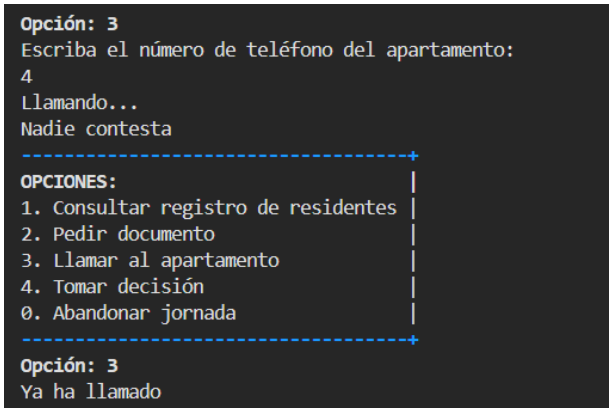


Figura 22. Llamada con teléfono incorrecto y llamada repetida. *Evidencia CP19, CP20.*



Figura 23. Visitante que vive solo: Cristina Casado. *Evidencia CP30.*

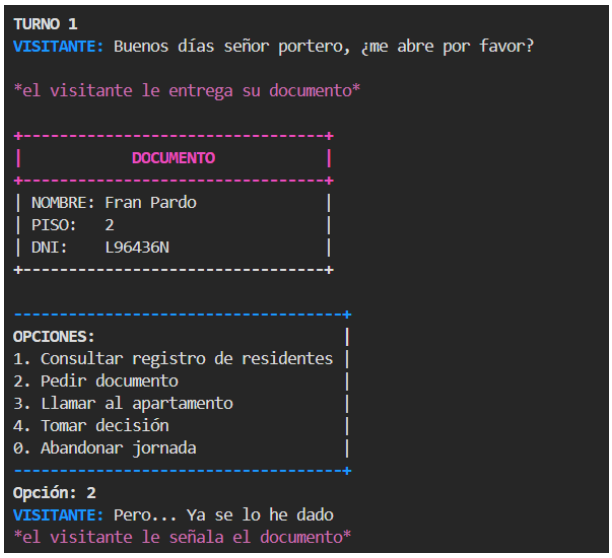


Figura 24. Pedir el documento por segunda vez. *Evidencia CP21.*

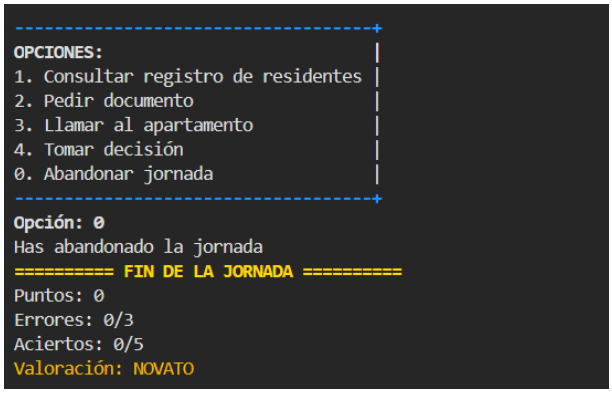


Figura 25. Abandonar la jornada en el primer turno. *Evidencia CP29.*

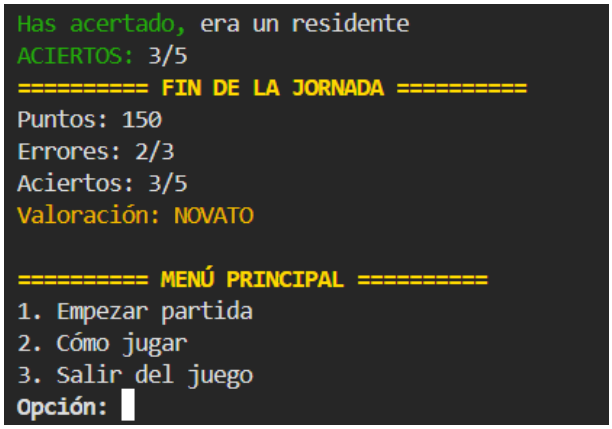


Figura 26. Dos errores y tres aciertos: 300 - 50 - 100 = 150 puntos. *Evidencia CP28.*

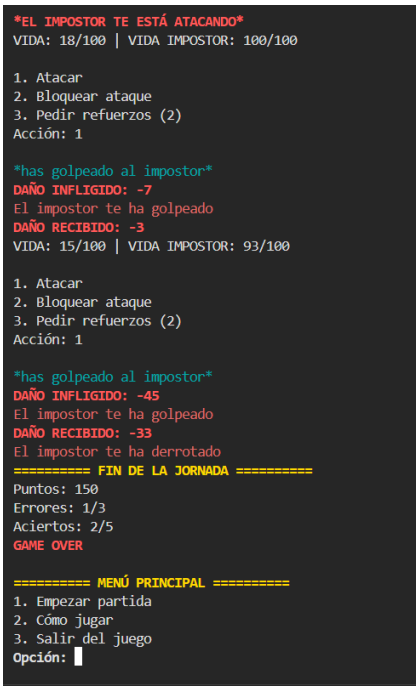


Figura 27. Duelo perdido: «El impostor te ha derrotado» y GAME OVER. *Evidencia CP26.*